

Lambdas

Ripasso sulle interfacce

Un'**interfaccia** in Java definisce un *contratto*: dice **cosa** una classe può fare, senza dire **come** lo fa. Quando una classe la implementa (parola chiave `implements`), "firma" quel contratto e si impegna a rispettarlo. Se la classe è concreta (cioè istanziabile), il compilatore la obbliga a fornire il codice per **ogni metodo astratto** dell'interfaccia.

Regole chiave ripassate nella lezione:

- Una classe può **estendere una sola classe** (`extends`), ma può **implementare molte interfacce**. Esempio: `class Dog extends Animal implements Movable, Lovable` → un cane è contemporaneamente un animale, movable e lovable.
- Da **Java 8** in poi le interfacce possono avere metodi concreti: i metodi `default` e `static`. Le classi che implementano l'interfaccia **non sono obbligate** a reimplementarli.
- I metodi `default` sono **ereditabili**; i metodi `static` **no**.
- Le interfacce sono **implicitamente astratte** (anche se ometti la parola `abstract`).
- Tutti i metodi di un'interfaccia sono **implicitamente public**; tutte le variabili sono **public static final** (cioè costanti).
- Non puoi istanziare direttamente un'interfaccia (come per le classi astratte).

Interfaccia funzionale (regola SAM)

Definizione: un'**interfaccia funzionale** è un'interfaccia che ha **un solo metodo astratto**. La regola si chiama **SAM** = *Single Abstract Method*.

Cosa **non conta** nel conteggio del "singolo metodo astratto":

- i metodi `default`,
- i metodi `static`,
- i metodi ereditati da `Object` (es. `toString`, `equals`).

Esempio minimo:

```
interface FI {  
    void m () //unico metodo astratto → interfaccia funzionale valida  
}
```

Che cos'è una lambda

Definizione: un'espressione **lambda** è un blocco di codice conciso che rappresenta **un'istanza di una classe che implementa un'interfaccia funzionale**, ridotta all'essenziale.

Punti fondamentali spiegati:

- Una lambda **funziona solo con le interfacce funzionali**. Il motivo: il compilatore deve sapere *quale* metodo stai implementando. Se ci fosse più di un metodo astratto, non potrebbe decidere. Con **un solo** metodo astratto, il compilatore sa subito di quale si tratta.
- Le lambda assomigliano a metodi e a volte sono chiamate "**metodi anonimi**", ma tecnicamente sono **istanze** di classi che implementano l'interfaccia funzionale. Il compilatore fa molto lavoro "in background".
- Il compilatore **deduce** quasi tutto dalla definizione dell'interfaccia (che ha un solo metodo astratto).

Esempio: Prima e dopo Java 8

Data l'interfaccia funzionale

```
interface I {  
    void m ();  
}
```

Per-Java 8 - Classe interna anonima:

```
I ia = new I () {  
    @Override  
    public void m(){  
        System.out.println("I::m");  
    }  
};  
ia.m();           // qui avviene l'esecuzione
```

Qui `new I() { ... }` **non** istanzia l'interfaccia: le parentesi graffe contengono il **codice della classe** (anonima) che la implementa, sovrascrivendo `m()`.

con Java 8 – lambda:

```
I lambdaI = () -> {  
    System.out.println("versione lambda");  
};  
I lambdaI2 = () -> System.out.println("versione lambda");  
  
lambdaI.m();           // esegue la lambda  
lambdaI2.m();          // esegue la lambda
```

Come leggere una lambda:

- Le **parentesi tonde ()** a sinistra corrispondono alle parentesi del metodo `void m()`: dichiarano gli **input** (qui nessuno).
- La **freccia ->** separa gli input (sinistra) dal codice del metodo (destra).
- A destra: con le **graffe { }** puoi mettere un blocco di più istruzioni (serve il `;` interno); se hai **una sola istruzione**, puoi **omettere le graffe**.
- La lambda **definisce** solo il codice: non viene eseguito finché non chiami il metodo (`.m()`).

Lambda con un'interfaccia funzionale personalizzata

Qui viene creata un'interfaccia funzionale **generica** fatta a mano, per poi mostrare che è quasi identica a `Predicate` dell'API.

```
interface Evaluate<T> {           // <T> = generics: funziona per qualsiasi tipo  
    boolean isNegative(T t);      // unico metodo astratto → interfaccia funzionale  
}
```

Il `<T>` (generics) permette di riutilizzare la stessa interfaccia per interi, stringhe, cani, ecc. — dichiarando il tipo al momento dell'uso, l'interfaccia lavorerà **solo** per quel tipo. (I generics non sono argomento d'esame, ma servono qui.)

Uso con gli interi:

```
Evaluate<Integer> lambda = i -> i < 0;  
  
lambda.isNegative(-1); // true → qui avviene l'esecuzione  
lambda.isNegative(1);  // false
```

Spiegazioni:

- `i` è l'**argomento** in ingresso del metodo `isNegative`. Con **un solo parametro** puoi omettere le parentesi attorno a `i`.
- `i < 0` restituisce un `boolean`, quindi il compilatore inserisce automaticamente il `return`: non serve scriverlo.
- Se invece scrivi un **blocco con graffe**, allora devi mettere esplicitamente `return` e il `;`:

```
Evaluate<Integer> lambda = i -> { return i < 0; };
```

- La definizione della lambda **non** esegue nulla. L'esecuzione avviene solo quando chiami `lambda.isNegative(...)`.

Uso dei Predicate

Definizione — Predicate<T>: interfaccia funzionale predefinita dell'API (`java.util.function`). Ha un solo metodo astratto **test**, che prende **un** argomento del tipo `T` e restituisce un **boolean**. È di fatto la versione "ufficiale" dell'`Evaluate` visto prima.

```
Predicate<Integer> predicate = i -> i < 0;
```

```
predicate.test(-1); // true
```

```
predicate.test(1); // false
```

Uso dentro un metodo generico

```
static <T> void check(T t, Predicate<T> predicate) {  
    System.out.println(predicate.test(t));  
}
```

- `<T>` prima del tipo di ritorno segna il metodo come **generico**: `T` sta per "un tipo qualsiasi", collegato a runtime.
- Dentro, chiama `predicate.test(t)`, che restituisce il `boolean`.

Chiamate con interi (test "è pari"):

```
int x = 4;
```

```
check(x, n -> n % 2 == 0); // 4 % 2 == 0 → true (pari)
```

`n % 2` (modulo) dà il **resto** della divisione per 2: se è 0, il numero è pari. Nota che il parametro può chiamarsi come vuoi (`n`), perché ha **scope locale alla lambda**.

Il bello dei generics: lo **stesso** metodo `check` funziona con un **tipo diverso**, ad esempio le stringhe:

```
String name = "Mr. Joe Bloggs";  
check(name, s -> s.startsWith("Mr.)); // true  
check("Ms. Bloggs", s -> s.startsWith("Mr.)); // false
```

Qui `s` è una `String`; il compilatore capisce dal contesto che il `Predicate` è tipizzato per `String`, quindi puoi chiamare `startsWith` (metodo delle stringhe) che restituisce `true/false`.

Predicate e BiPredicate (API)

Predicate<T>

Ripasso: genericamente tipizzato, metodo `test` che prende **un** parametro e restituisce `boolean`.

```
Predicate<String> p = s -> s.contains("city") || s.contains("City");  
p.test("Vatican City"); // true → contiene "City"
```

Con un solo parametro le parentesi sono opzionali; essendo un'espressione semplice, niente graffe e niente `return`.

BiPredicate<T, U>

Definizione: come `Predicate`, ma il metodo `test` prende **due** input (T e U) e restituisce `boolean`. (Il "**Bi**" richiama il due — come le due ruote di una bicicletta.)

```
BiPredicate<String, Integer> bp = (str, len) -> str.length() == len;  
bp.test("Vatican City", 12); // "Vatican City" ha lunghezza 12 → nell'esempio il confronto risulta false
```

Con **due** parametri servono **le parentesi** attorno agli argomenti e la **virgola** che li separa.

Supplier (Fornitore)

Definizione — Supplier<T>: interfaccia funzionale che **fornisce** un valore **senza ricevere alcun input**. Metodo astratto `get()`, che non prende nulla e **restituisce** un valore di tipo T (un tipo, non un primitivo).

Esempi:

```
Supplier<StringBuilder> sb = () -> new StringBuilder();  
sb.get().append("SK"); // get() esegue e restituisce lo StringBuilder, poi append  
  
Supplier<LocalTime> time = () -> LocalTime.now();  
time.get(); // restituisce l'ora locale
```

```
Supplier<Double> rand = () -> Math.random();
rand.get();           // numero double, 0.0 incluso fino a < 1.0
```

Note:

- Le parentesi () a sinistra sono **vuote**: `get` non prende niente.
- Se cambiassi il tipo del `Supplier<Double>` in `String`, avresti un **errore di compilazione**, perché `Math.random()` restituisce un `double`: il tipo dichiarato e quello restituito devono corrispondere.

Consumer e BiConsumer

Definizione — Consumer<T>: interfaccia funzionale che **consuma** un valore: prende qualcosa in ingresso e **non restituisce nulla** (`void`). Metodo astratto **`accept(T t)`**.

```
Consumer<String> printer = s -> System.out.println(s);
printer.accept("ciao"); // stampa "ciao", non restituisce nulla
```

Uso tipico — iterare una lista con `forEach` (che accetta proprio un `Consumer`):

```
List<String> names = new ArrayList<>(List.of("John", "Mary"));
names.forEach(s -> System.out.println(s)); // stampa John, poi Mary
```

`forEach` scorre ogni elemento e chiama il `Consumer` su ciascuno.

Definizione — BiConsumer<T, U>: come `Consumer`, ma `accept` prende **due** argomenti e restituisce sempre `void`. Utilissimo per le **mappe** (chiave + valore).

```
Map<String, String> capitalCities = new HashMap<>();
BiConsumer<String, String> putIt = (k, v) -> capitalCities.put(k, v);
putIt.accept("Dublin", "Ireland");
putIt.accept("Washington D.C.", "US");
```

Dettaglio interessante: `Map.put` **restituirebbe** il valore precedente (o `null`), ma dentro il `BiConsumer` quel valore di ritorno viene **ignorato** — è come chiamare un metodo scartandone il ritorno. `put` viene comunque eseguito.

Decorare l'output iterando la mappa con `forEach` (che sulle mappe accetta un `BiConsumer`):

```
BiConsumer<String, String> pretty = (k, v) -> System.out.println(k + " is the capital of " + v);
capitalCities.forEach(pretty);
```

```
// Dublin is the capital of Ireland
```

```
// Washington D.C. is the capital of US
```

`forEach` funziona sia sulle liste (con `Consumer`) sia sulle mappe (con `BiConsumer`).

Function e BiFunction

Definizione — Function<T, R>: riceve **un input** (T) e restituisce **un output** (R). Input e output possono essere dello **stesso** tipo o **diversi**. Metodo astratto **apply**. Il tipo di ritorno è **sempre l'ultimo** parametro generico.

```
Function<String, Integer> len = s -> s.length();  
len.apply("Moscow"); // 6 (input String, output Integer)
```

Definizione — BiFunction<T, U, R>: riceve **due input** (T, U) e restituisce **un output** (R); il tipo di ritorno resta **l'ultimo** specificato.

```
BiFunction<String, String, Integer> sumLen = (a, b) -> a.length() + b.length();  
sumLen.apply("William", "Shakespeare"); // 7 + 11 = 18
```

Altro esempio con tre stringhe (input, input, output):

```
BiFunction<String, String, String> concat = (a, b) -> b + a;  
concat.apply("William", "Shakespeare"); // "ShakespeareWilliam"
```

UnaryOperator e BinaryOperator

Queste sono **specializzazioni** di Function e BiFunction per il caso in cui **input e output sono dello stesso tipo**.

- Operazione **unaria** = su un solo operando (es. incrementare di 1).
- Operazione **binaria** = su due operandi (es. sommare, moltiplicare, dividere due numeri).

Definizione — UnaryOperator<T>: è una Function in cui **input e output hanno lo stesso tipo T**. apply(T) restituisce T. Basta specificare **un solo** tipo generico.

```
UnaryOperator<String> greet = name -> "My name is " + name;  
greet.apply("Shawn"); // "My name is Shawn"
```

Mentre in una Function i tipi possono differire, in UnaryOperator **devono corrispondere**.

Definizione — BinaryOperator<T>: è una BiFunction in cui **i due input e l'output** hanno tutti lo stesso tipo T. Versione più breve/concisa di BiFunction<T, T, T>.

```
BinaryOperator<String> join = (a, b) -> b + a;  
join.apply("William", "Shakespeare"); // "ShakespeareWilliam"
```

final ed "effectively final"

Il concetto: una lambda definita dentro un metodo può **usare** le variabili locali di quel metodo, ma **solo** se sono **final** o **"effectively final"** (effettivamente finali).

Distinzioni:

- Una **variabile di istanza** (campo dell'oggetto) puoi cambiarla liberamente — ti basta un'istanza per accedervi. Nessun problema.
- Una **variabile locale** del metodo, invece, se viene usata dentro una lambda, **non può più essere modificata**.

Definizione — **"effectively final"**: una variabile che è finale **in tutto tranne che nel nome**, cioè non hai scritto la parola `final`, ma di fatto **non la modifichi mai**. Il compilatore la tratta come finale.

```
int x = 12;           // locale, mai modificata → effectively final
```

```
Predicate<Integer> p = n -> n < x; // la lambda "cattura" x (snapshot con valore 12)
```

```
// x = 20; → ERRORE di compilazione: x è usata dalla lambda, deve restare final
```

Perché questa regola: la lambda **scatta una "istantanea" (snapshot)** della variabile locale nel momento in cui viene definita. Una lambda è un pezzo di codice che può essere **passato in giro** (a un filtro dati, ad altri oggetti) ed eseguito **più tardi**. Perché tutto funzioni, il metodo circostante e la lambda devono avere **sempre lo stesso valore** di `x`. Quindi `x` non può cambiare:

- **né dentro** la lambda,
- **né dopo** nel metodo (nemmeno prima o dopo la definizione della lambda, una volta che viene catturata).

Dettaglio: l'errore di compilazione appare **sulla lambda**, indipendentemente dal fatto che la modifica avvenga dentro la lambda o nel metodo esterno. Se invece **rimuovi l'uso di x** dalla lambda, torni a poter modificare `x` liberamente. Vale per qualsiasi lambda, non solo per i `Predicate`.

Riferimenti ai metodi (Method References)

Idea: le lambda rendono il codice più conciso; i **method reference** possono renderlo **ancora più** conciso. Si usano quando la lambda **si limita a chiamare un metodo** e il parametro viene semplicemente **passato** a quel metodo — eliminando la **ridondanza** di scrivere il parametro due volte.

Esempio (da `Consumer<String>`):

```
List<String> names = new ArrayList<>(List.of("Shawn", "Mary", "John"));
```

```
// versione lambda
```

```
names.forEach(s -> System.out.println(s));
```

```
// versione method reference (identica nel comportamento)
```

```
names.forEach(System.out::println);
```

Sintassi: **doppio due punti ::** e **nessuna parentesi tonda** dopo il nome del metodo.

I quattro tipi di method reference: bound (vincolato), unbound (non vincolato), static, constructor.

- **Bound:** conosci l'istanza al momento della compilazione.
- **Unbound:** l'istanza da usare viene fornita **a runtime**.

Il **contesto** (cioè *quale* interfaccia funzionale stai creando) è **fondamentale**: è ciò che permette al compilatore di trasformare, in background, il method reference nella lambda equivalente. Per questo negli esempi seguenti viene sempre mostrata **sia** la versione lambda **sia** quella con method reference.

Bound method references (riferimenti vincolati)

Bound: il riferimento è legato a un'istanza **già nota** (una variabile specifica).

Esempio con `Supplier`:

```
String name = "Mr. George Bloggs";
```

```
// lambda (suffisso L)
```

```
Supplier<String> supL = () -> name.toLowerCase();
```

```
// method reference (suffisso MR) — vincolato all'istanza "name"
```

```
Supplier<String> supMR = name::toLowerCase;
```

```
supL.get(); // "mr. george bloggs"
```

```
supMR.get(); // "mr. george bloggs"
```

Non serve specificare l'istanza al momento di chiamare `get()`, perché il riferimento è **già legato** all'istanza `name`.

Esempio con `Predicate` (metodo che prende un argomento):

```
Predicate<String> pL = s -> name.startsWith(s);
```

```
Predicate<String> pMR = name::startsWith;
```

```
pMR.test("Mr."); // true
pMR.test("Ms."); // false
```

Punto importante sul **contesto**: `startsWith` è **sovraccaricato** (overloaded) nella classe `String` — esiste una versione con **una** stringa e una con **stringa + indice**. Poiché stai costruendo un `Predicate` (metodo `test` con **un solo** parametro), il compilatore sceglie la versione che prende **un** argomento, senza generare errori. Senza il contesto, non saprebbe quale usare.

Unbound method references (riferimenti non vincolati)

Unbound: non legghi il riferimento a un'istanza specifica; l'istanza arriva **a runtime**, e sarà **il primo parametro** passato.

Esempio con `Function<String, String>`:

```
Function<String, String> fL = s -> s.toUpperCase();
Function<String, String> fMR = String::toUpperCase; // unbound

fMR.apply("Sean"); // "SEAN"
```

Qui specifichi il **tipo** `String` e il metodo, senza istanza e senza parentesi. A runtime, la stringa passata ("Sean") diventa **l'istanza** su cui viene eseguito `toUpperCase`.

Esempio più complesso con `BiFunction<String, String, String>` (`concat`):

```
BiFunction<String, String, String> bfL = (s1, s2) -> s1.concat(s2);
BiFunction<String, String, String> bfMR = String::concat; // unbound

bfMR.apply("Shaun", "Kennedy"); // "ShaunKennedy"
```

Regola chiave dell'unbound con più parametri: **il primo parametro** è usato come **istanza** su cui eseguire il metodo; **tutti gli altri** (secondo, terzo, ecc.) sono passati come **argomenti** del metodo. Quindi `"Shaun".concat("Kennedy")` → il primo (Shaun) è l'istanza, il secondo (Kennedy) è l'argomento.

Static method references

Definizione: un riferimento a un **metodo statico**. Sono considerati **unbound**. Esempio tipico: `Collections.sort(list)`.

```
Consumer<List<Integer>> cL = list -> Collections.sort(list); // lambda
Consumer<List<Integer>> cMR = Collections::sort; // static method reference
```

```
List<Integer> numbers = new ArrayList<>(List.of(5, 2, 8, 1));
cL.accept(numbers); // ordina la lista
System.out.println(numbers);
```

```
List<Integer> numbers2 = new ArrayList<>(List.of(9, 3, 7));
cMR.accept(numbers2); // ordina la lista (versione method reference)
System.out.println(numbers2);
```

Come lavora il compilatore qui: specifichi `Collections::sort`, che è un metodo **statico**. Dal **contesto** (un `Consumer`, il cui `accept` prende **un** argomento) il compilatore capisce che si tratta della versione di `sort` che prende **solo la lista** (e non quella `sort(list, comparator)` con due argomenti). La lista verrà passata più tardi con `accept`. Guardare la lambda equivalente aiuta a "vedere" cosa succede in background: passi qualcosa dentro, la vedi entrare e capisci il risultato (lista ordinata).

Constructor method references

Definizione: tipo speciale di method reference che usa la parola chiave **new** al posto del nome di un metodo, per **istanziare un oggetto**. I `Supplier` sono molto usati con questi riferimenti.

Esempio con `Supplier` (costruttore senza argomenti):

```
Supplier<StringBuilder> sbL = () -> new StringBuilder(); // lambda
Supplier<StringBuilder> sbMR = StringBuilder::new; // constructor reference

sbL.get().append("...");
sbMR.get().append("...");
```

Dal contesto (un `Supplier`: niente input, restituisce qualcosa) il compilatore capisce di dover creare uno `StringBuilder` con `new`.

Se invece devi **passare un argomento** al costruttore e **ricevere** qualcosa indietro, serve una **Function**:

```
// ArrayList ha un costruttore che prende la capacità iniziale (int)
Function<Integer, List<String>> fL = x -> new ArrayList<>(x); // lambda
Function<Integer, List<String>> fMR = ArrayList::new; // constructor reference

fL.apply(10); // crea un ArrayList con capacità iniziale 10
fMR.apply(5); // crea un ArrayList con capacità iniziale 5
```

Anche qui la lambda mostra la ridondanza (x a sinistra e a destra), che il constructor reference elimina: basta dire **quale tipo** vuoi (ArrayList) e usare new. Il metodo apply è il metodo funzionale della Function.

L'importanza del contesto per i method reference

Questo è il concetto riassuntivo: **lo stesso identico method reference** può comportarsi in modo **diverso** a seconda del **contesto**, cioè di **quale interfaccia funzionale** stai creando sul lato sinistro (Supplier? Function? BiFunction?). È il contesto a dire al compilatore: qual è il metodo funzionale? qual è l'input? qual è l'output?

Classe di esempio:

```
class Person {  
    static int howMany(Person... people) { // varargs: zero o più Person  
        return people.length;  
    }  
}
```

Lo stesso riferimento Person::howMany viene usato in **tre** contesti diversi:

1) Come Supplier<Integer> — nessun input:

```
Supplier<Integer> supL = () -> Person.howMany(); // lambda  
Supplier<Integer> supMR = Person::howMany;      // stesso method reference  
  
supMR.get(); // 0 → nessuna Person passata, length = 0
```

Il Supplier non ha input, quindi howMany viene chiamato **senza** persone → 0.

2) Come Function<Person, Integer> — un input:

```
Function<Person, Integer> fL = p -> Person.howMany(p); // lambda  
Function<Person, Integer> fMR = Person::howMany;      // stesso method reference  
  
fMR.apply(new Person()); // 1
```

Dal contesto il compilatore sa che entra **una** Person, che verrà passata a howMany → 1.

3) Come BiFunction<Person, Person, Integer> — due input:

```
BiFunction<Person, Person, Integer> bfL = (p1, p2) -> Person.howMany(p1, p2); // lambda  
BiFunction<Person, Person, Integer> bfMR = Person::howMany; // stesso method  
reference
```

```
bfMR.apply(new Person(), new Person()); // 2
```

Due Person in ingresso → howMany le riceve entrambe → 2.

Output complessivo dei tre casi: 0, 0 — 1, 1 — 2, 2 (ogni coppia = versione lambda + versione method reference).

La lezione fondamentale: il **method reference Person : : howMany** è scritto in modo identico in tutti e tre i casi, ma **ciò che accade in background è completamente diverso**, ed è determinato **dal contesto**:

- il `Supplier` non ha input → `howMany` non riceve nulla;
- la `Function` ha un input → `howMany` riceve quell'argomento;
- la `BiFunction` ha due input → `howMany` li riceve entrambi.

Per questo, quando un method reference è difficile da capire, il trucco è **mapparlo mentalmente alla lambda equivalente**: diventa subito chiaro cosa il compilatore sta facendo dietro le quinte.
